

Algorithms for Collaborative Harmonization- Supplementary Material

Eyal Briman, Eyal Leizerovich, and Nimrod Talmon

Ben-Gurion University of the Negev, Israel

1 Appendix

1.1 Complexity Proofs

We begin with Plurality; then go on to Kemeny, PAV, and clustered Kemeny.

Complexity of Plurality

Observation 1. *Plurality can be computed in polynomial-time.*

Proof. This follows as Plurality treats each chord-position independently. Formally, the following algorithm has a polynomial time complexity: For every column B_j , the algorithm selects the chord that appears the most.

Also the 2-gram version of Plurality admits a polynomial-time algorithm.

Theorem 1. *Plurality with 2-gram can be computed in polynomial time.*

Proof. This proof follows a structure similar to the previous theorem, where we establish that $T(1, a)$ is equivalent to Plurality without the 2-gram feature, making it a polynomial algorithm.

Complexity of Kemeny Next, we show that also Kemeny is polynomial-time solvable. This is perhaps surprising (as the original Kemeny voting rule that corresponds to minimizing the sum of swap distance in ordinal elections in NP-hard [2]); it is so, however, as our adaptation of Kemeny also considers each chord-position separately.

Observation 2. *Kemeny can be computed in polynomial time.*

Proof. We describe a polynomial-time algorithm: For every column B_j and for every chord, the algorithm selects the chord that minimizes the summation of distances considering all agents. This results in a complexity of $O(m \cdot n \cdot k)$, which is polynomial.

Furthermore, even Kemeny with 2-gram can be computed in polynomial time; here, however, we use dynamic programming that follows the chord progression.

Theorem 2. *Kemeny with 2-gram can be computed in polynomial time.*

Proof. We establish the polynomial nature of Kemeny with 2-gram by describing an algorithm that is based on dynamic programming. Let $T(j, a)$ denote the minimal cost of Kemeny with 2-gram for the first j chords, given that the last chord is $W[j] = a$, where a can be any of the m chords.

To begin, we note that $T(1, a)$ corresponds to the case where there is only one chord, essentially Kemeny without the 2-gram feature. This base case has been previously shown to be polynomial.

Next, we introduce a recurrence relation: $T(j, a) = \arg \min_{a'} [T(j-1, a') + \sum_{i=1}^n d(b_{i,j}, a) - \log(p(a', a))]$. This recurrence efficiently computes the minimal cost for each chord a for the first j chords and thus, also for the total of k chords.

Since we can find $\arg \min_a T(k, a)$ in polynomial time, this algorithm's overall complexity is polynomial. This process is akin to finding the optimal solution W in polynomial time.

Complexity of PAV

Theorem 3. *PAV and PAV with 2-gram are NP-hard.*

Proof. We establish the NP-hardness of the Proportional algorithm by considering the general utility function U , which returns a vector of utilities for each agent i where $U[j] \in [0, 1]$ for all $j \in [k]$. We also note that the Proportional algorithm encompasses Proportional Approval Voting (PAV) as a special case, where $U[i] \in \{0, 1\}$. This establishes that Proportional is NP-hardness [3].

By considering the special case of setting x_p to 1, which is equivalent to the Proportional algorithm, we can conclude that Proportional with 2-gram is also NP-hard.

Complexity of Clustered Kemeny We proceed to consider Clustered-Kemeny.

Theorem 4. *Clustered-Kemeny and Clustered-Kemeny with 2-gram are NP-hard.*

Proof. To establish the NP-hardness of Clustered-Kemeny and Clustered-Kemeny with 2-gram, we start by drawing a connection to the k -median problem, proven as NP-Hard [1]. In our case, we consider the k -string median problem since a chord sequence effectively represents a string.

The k -string median problem is formally defined as follows: Given a set of n strings $S = \{s_1, s_2, \dots, s_n\}$ of length ℓ , a string distance metric d normalized to return a value between 0 and 1, and a threshold t , determine if there exist k median strings $M = \{m_1, m_2, \dots, m_k\}$ of length ℓ each such that:

$$\sum_{i=1}^n \min_{1 \leq j \leq k} d(s_i, m_j) \leq t .$$

Given such an instance, we construct an instance to the decision variant of Clustered-Kemeny that is given the partitioning into sections, as well as defining

lengths for each section is formally represented by:

$$\sum_{z \in Z} \sum_{i=1}^n \sum_{j=1}^k P(j, z, Z) \cdot Q(i, z, Z) \cdot (d(b_{i,j}, W[j])) < t .$$

To construct an instance for the k-median problem comprising n strings of length ℓ , we create a corresponding instance for the Kemeny-Clustering problem by transforming each input string into an agent representation. In this transformation, each string is replicated k times to generate the agents for the Kemeny-Clustering instance (e.g., an input string "abc" replicated $k = 3$ times transforms into an agent with the string "abcabcabc"). We then set $value = 0$ for each agent i in $Q(i, z, Z)$, and a partition into k sections by indices is defined as $Z = \{L, L \cdot 2, L \cdot 3, \dots, L \cdot k\}$.

It is evident that the k-median problem is a yes instance if and only if the Kemeny-Clustering problem is a yes instance. This implies that even when the given partition and length are specified, the Clustered-Kemeny problem remains challenging, as it necessitates checking multiple possible partitions for each number of sections, thereby intensifying the complexity of optimization.

Furthermore, when considering Clustered-Kemeny with 2-gram, the methodology is similar to the hardness proof of Proportional with 2-gram, achieved by setting $x_{KC} = 1$.

1.2 Simulations Full Result

Next We provide the full results tables below.

1.3 Further Analysis of the Simulation Results

We provide some further analysis of the simulation results.

Distance Evaluation

- **Plurality and Plurality + 2-gram** consistently exhibit significantly lower Jaccard Distances, indicating their ability to maintain closer proximity to the original sequences, especially evident as the errors increase.
- **Kemeny and Kemeny + 2-gram** perform competitively, showcasing slightly higher Jaccard Distances compared to the Plurality-based algorithms but demonstrating resilience against errors in chord alterations.
- **PAV methods** generally showcase stable but relatively higher Jaccard Distances compared to Plurality and Kemeny algorithms, implying slightly inferior performance in maintaining sequence proximity.

Table 1: Errors (0,1) - (3,4) and 8 agents - Algorithms Scores.

Error	Algorithm	Song Similarity	Cluster Coherence	Musical Coherence
(0,1)	Plurality	0.41	7.1	12
(0,1)	Plurality + 2-gram	0.31	7.1	13
(0,1)	Kemeny	0.46	7.1	11
(0,1)	Kemeny + 2-gram	0.39	7.1	11
(0,1)	PAV	0.41	7.1	12
(0,1)	PAV + 2-gram	0.41	7.1	12
(0,1)	Clustered-Kemeny	1	7.3	11
(0,1)	Clustered-Kemeny + 2-gram	0.073	7	13
(1,2)	Plurality	26	29	0.18
(1,2)	Plurality + 2-gram	23	28	0.31
(1,2)	Kemeny	22	27	0.12
(1,2)	Kemeny + 2-gram	21	27	0.3
(1,2)	PAV	26	29	0.18
(1,2)	PAV + 2-gram	26	29	0.18
(1,2)	Clustered-Kemeny	26	29	0.17
(1,2)	Clustered-Kemeny + 2-gram	26	29	0.18
(2,3)	Plurality	58	40	0.0025
(2,3)	Plurality + 2-gram	57	40	0.0066
(2,3)	Kemeny	52	37	0.0012
(2,3)	Kemeny + 2-gram	52	38	0.0052
(2,3)	PAV	58	40	0.0025
(2,3)	PAV + 2-gram	58	40	0.0025
(2,3)	Clustered-Kemeny	58	40	0.0024
(2,3)	Clustered-Kemeny + 2-gram	59	41	0.0031
(3,4)	Plurality	70	43	0.00083
(3,4)	Plurality + 2-gram	70	42	0.0024
(3,4)	Kemeny	67	39	0.00042
(3,4)	Kemeny + 2-gram	66	39	0.0018
(3,4)	PAV	70	43	0.00083
(3,4)	PAV + 2-gram	70	43	0.00085
(3,4)	Clustered-Kemeny	70	43	0.00082
(3,4)	Clustered-Kemeny + 2-gram	71	44	0.0011

Cluster Coherence Analysis

- **Plurality, Plurality + 2-gram** consistently maintain relatively low cluster distances across different error ranges and agent counts, indicating their ability to preserve coherence and smooth transitions within musical segments.
- **Clustered-Kemeny**, especially in scenarios with higher errors and agent counts, shows higher cluster distances, implying challenges in maintaining cohesion within smaller musical sections compared to other algorithms.

Musical Coherence

- **Plurality and Kemeny algorithms** generally showcase better Musical Coherence, indicating that the chord progressions they generate align more harmoniously with the provided 2-gram.

Table 2: Errors (0,1) - (3,4) and 16 agents - Algorithms Scores.

Error	Algorithm	Song Similarity	Cluster Coherence	Musical Coherence
(0,1)	Plurality	0.0012	7	13
(0,1)	Plurality + 2-gram	0.0015	7	13
(0,1)	Kemeny	0.004	7	12
(0,1)	Kemeny + 2-gram	0.0029	7	12
(0,1)	PAV	0.0012	7	13
(0,1)	PAV + 2-gram	0.0012	7	13
(0,1)	Clustered-Kemeny	0.36	7.1	12
(0,1)	Clustered-Kemeny + 2-gram	0.031	7	13
(1,2)	Plurality	8.9	7.2	7.2
(1,2)	Plurality + 2-gram	7.2	7.2	7.1
(1,2)	Kemeny	8.8	8.2	8.2
(1,2)	Kemeny + 2-gram	8.2	8.2	8.1
(1,2)	PAV	8.9	8.9	8.9
(1,2)	PAV + 2-gram	8.9	8.9	8.9
(1,2)	Clustered-Kemeny	9.1	8.9	8.9
(1,2)	Clustered-Kemeny + 2-gram	8.9	8.9	8.9
(2,3)	Plurality	44	42	42
(2,3)	Plurality + 2-gram	42	42	42
(2,3)	Kemeny	41	40	40
(2,3)	Kemeny + 2-gram	40	40	40
(2,3)	PAV	44	44	44
(2,3)	PAV + 2-gram	44	44	44
(2,3)	Clustered-Kemeny	44	45	45
(2,3)	Clustered-Kemeny + 2-gram	45	45	45
(3,4)	Plurality	66	66	66
(3,4)	Plurality + 2-gram	66	66	66
(3,4)	Kemeny	62	62	62
(3,4)	Kemeny + 2-gram	62	62	62
(3,4)	PAV	66	66	66
(3,4)	PAV + 2-gram	66	66	66
(3,4)	Clustered-Kemeny	66	67	67
(3,4)	Clustered-Kemeny + 2-gram	67	67	67

- **Plurality + 2-gram and Kemeny + 2-gram** approaches display competitive Musical Coherence, suggesting their effectiveness in creating chord progressions that fit well within the context of the provided 2-gram.

1.4 Toy Example

We provide a toy example for illustrating instances of the model and the aggregation algorithms used throughout the paper.

Example 1. Let us consider a simple scenario where 3 agents were tasked to create a sequence of 4 chords each:

- Agent 1: Cmaj7, Dm7, G7, Cmaj7.

Table 3: Errors (0,1) - (3,4) and 32 agents - Algorithms Scores with 2-grams included.

Error	Algorithm	Song Similarity	Cluster Coherence	Musical Coherence
(0,1)	Plurality	0	7	13
(0,1)	Plurality + 2-gram	0	7	13
(0,1)	Kemeny	0.0012	7	12
(0,1)	Kemeny + 2-gram	0.0009	7	12
(0,1)	PAV	0	7	13
(0,1)	PAV + 2-gram	0	7	13
(0,1)	Clustered-Kemeny	0.07	7.1	12
(0,1)	Clustered-Kemeny + 2-gram	0.015	7	13
(1,2)	Plurality	6.2	7.1	7.1
(1,2)	Plurality + 2-gram	5.1	7.1	7.1
(1,2)	Kemeny	6.1	7.2	7.2
(1,2)	Kemeny + 2-gram	5.8	7.2	7.2
(1,2)	PAV	6.2	7.1	7.1
(1,2)	PAV + 2-gram	6.2	7.1	7.1
(1,2)	Clustered-Kemeny	6.3	7.3	7.3
(1,2)	Clustered-Kemeny + 2-gram	6.1	7.1	7.1
(2,3)	Plurality	38	40	40
(2,3)	Plurality + 2-gram	37	40	40
(2,3)	Kemeny	35	38	38
(2,3)	Kemeny + 2-gram	35	38	38
(2,3)	PAV	38	40	40
(2,3)	PAV + 2-gram	38	40	40
(2,3)	Clustered-Kemeny	39	40	40
(2,3)	Clustered-Kemeny + 2-gram	38	40	40
(3,4)	Plurality	59	60	60
(3,4)	Plurality + 2-gram	60	60	60
(3,4)	Kemeny	54	55	55
(3,4)	Kemeny + 2-gram	55	55	55
(3,4)	PAV	59	60	60
(3,4)	PAV + 2-gram	59	60	60
(3,4)	Clustered-Kemeny	60	61	61
(3,4)	Clustered-Kemeny + 2-gram	60	61	61

- Agent 2: Am7, Dm7, E7, Am7.
- Agent 3: Cmaj7, Fmaj7, G7, Am7.

To illustrate collaborative harmonization algorithms, let's assess a proposed solution:

W : Cmaj7, Dm7, G7, Am7

- Plurality: This solution stands out as optimal since each chord was chosen by 2 out of the 3 agents.
- Kemeny: The given solution also appears optimal, demonstrating a minimal Jaccard distance of 2.167.

- PAV (Proportional Approval Voting): By computing PAV scores for each agent, we find: Agent 1: 0.5, Agent 2: 0.9166, Agent 3: 0.5. The overall PAV score sums up to 1.91666.
- Clustered Kemeny: Considering a maximum of 3 sections and setting *value* = 0 (where each agent is clustered into one section and the rest of their chord sequence is disregarded):
 - With 1 partition, the Kemeny score remains 2.167.
 - With 2 partitions, viable partitions include:
 - * First section: (Cmaj7, Dm7), Second section: (G7, Am7).
 - * First section: (Cmaj7), Second section: (Dm7, G7, Am7).
 - * First section: (Cmaj7, Dm7, G7), Second section: (Am7).
 - * First section: (Cmaj7), Second section: (Am7, Dm7, G7).
 - With 3 partitions, feasible partitions include:
 - * First section: (Cmaj7, Dm7), Second section: (G7), Third section (Am7).
 - * First section (Cmaj7), Second section (Dm7, G7), Third section (Am7).
 - * First section (Cmaj7), Second section (Dm7), Third section (G7, Am7).

The minimal distance score of 0 emerges with the partition of: First section (Cmaj7), Second section (Dm7, G7), Third section: (Am7). The clustering of agents would be Agent 3 in the first section, Agent 1 in the second section, and Agent 2 in the third section.
- In algorithms that manipulate the 2-gram score $G(W)$ through addition or reduction, we compute $G(W)$ utilizing the 2-gram probabilities:

$$G(W) = -(\log(CMaj7- > Dmin7) + \log(Dmin7- > G7) + \log(G7- > A-7)) = -(\log(0.0252903) + (0.199777) + \log(0.0053198)) = 10.524207$$

2 Figures

Example 2. Figure 2 is an example of an input and an output of our algorithms; the example contains 5 agents (depicted on the left) and a possible aggregated harmonic sequence (depicted on the right).

References

1. Charikar, M., Guha, S., Tardos, É., Shmoys, D.B.: A constant-factor approximation algorithm for the k-median problem. J. Comput. Syst. Sci. **65**(1), 129–149 (2002). <https://doi.org/10.1006/JCSS.2002.1882>, <https://doi.org/10.1006/jcss.2002.1882>
2. Hemaspaandra, E., Spakowski, H., Vogel, J.: The complexity of kemeny elections. Theor. Comput. Sci. **349**(3), 382–391 (2005). <https://doi.org/10.1016/J.TCS.2005.08.031>, <https://doi.org/10.1016/j.tcs.2005.08.031>
3. Skowron, P., Faliszewski, P., Lang, J.: Finding a collective set of items: From proportional multirepresentation to group recommendation. Artif. Intell. **241**, 191–216 (2016). <https://doi.org/10.1016/J.ARTINT.2016.09.003>, <https://doi.org/10.1016/j.artint.2016.09.003>

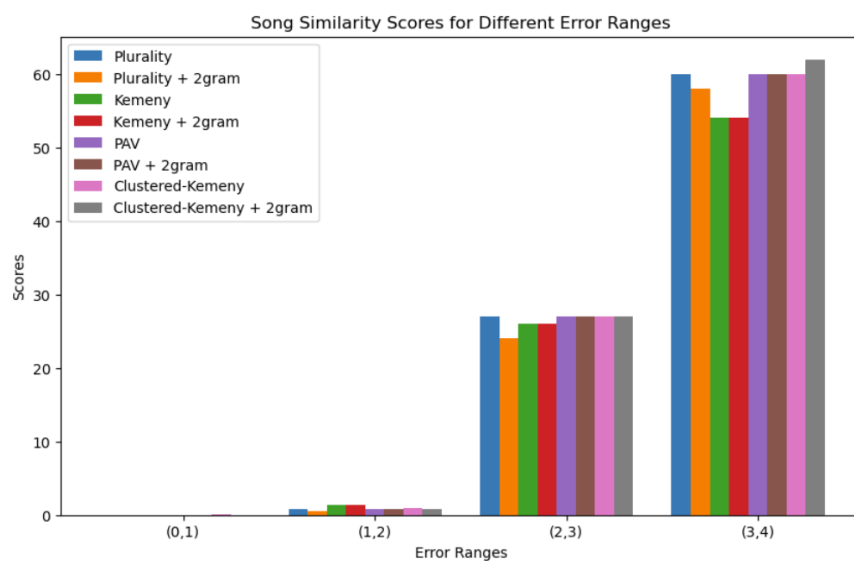
CMaj7, Cm7, CmMaj7, C7, Cdim-
Maj7, Cdim7, Cm7b5, Cm6, C+7,
C+maj7,
DbMaj7, Dbm7, DbmMaj7, Db7,
DbdimMaj7, Dbdim7, Dbm7b5,
Dbm6, Db+7, Db+maj7,
DMaj7, Dm7, DmMaj7, D7,
DdimMaj7, Ddim7, Dm7b5, Dm6,
D+7, D+maj7,
EbMaj7, Ebm7, EbmMaj7, Eb7,
EbdimMaj7, Ebdim7, Ebm7b5,
Ebm6, Eb+7, Eb+maj7,
Emaj7, Em7, EmMaj7, E7, Ed-
imMaj7, Edim7, Em7b5, Em6,
E+7, E+maj7,
FMaj7, Fm7, FmMaj7, F7, Fdim-
Maj7, Fdim7, Fm7b5, Fm6, F+7,
F+maj7,
GbMaj7, Gbm7, GbmMaj7, Gb7,
GbdimMaj7, Gbdim7, Gbm7b5,
Gbm6, Gb+7, Gb+maj7,
GMaj7, Gm7, GmMaj7, G7,
GdimMaj7, Gdim7, Gm7b5, Gm6,
G+7, G+maj7,
AbMaj7, Abm7, AbmMaj7, Ab7,
AbdimMaj7, Abdim7, Abm7b5,
Abm6, Ab+7, Ab+maj7,
AMaj7, Am7, AmMaj7, A7, Adim-
Maj7, Adim7, Am7b5, Am6, A+7,
A+maj7,
BbMaj7, Bbm7, BbmMaj7, Bb7,
BbdimMaj7, Bbdim7, Bbm7b5,
Bbm6, Bb+7, Bb+maj7,
BMaj7, Bm7, BmMaj7, B7, Bdim-
Maj7, Bdim7, Bm7b5, Bm6, B+7,
B+maj7.

Fig. 1: The alphabet of chords used in this study.

Figure 2 illustrates an instance to the collaborative harmonization problem. On the left, five staves of music are shown, each with a sequence of chords. A blue arrow points from this multi-staff sequence to a single staff on the right, representing an aggregated harmonic sequence.

Fig. 2: An example of an instance to the collaborative harmonization problem (on the left) with one possible aggregated harmonic sequence (on the right).

Fig. 3: Song Similarity vs Error Ranges for 32 Agents.



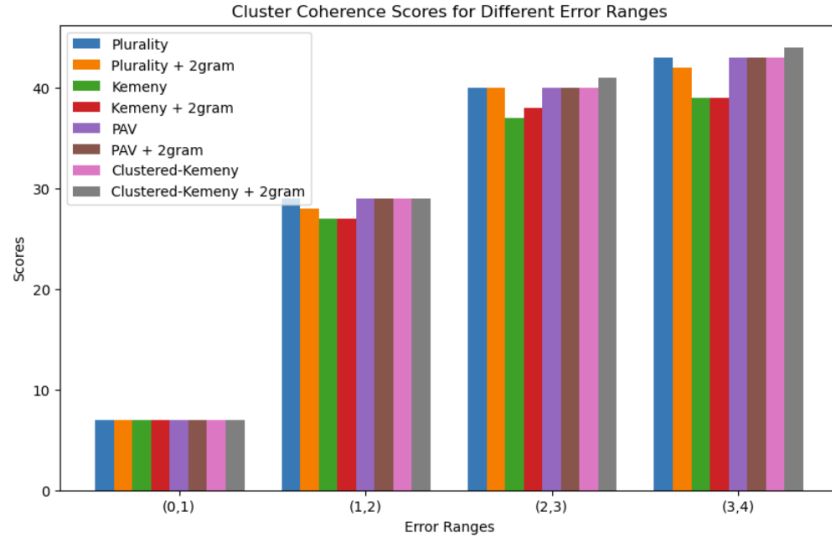


Fig. 4: Cluster Coherence vs Error Ranges for 32 Agents.

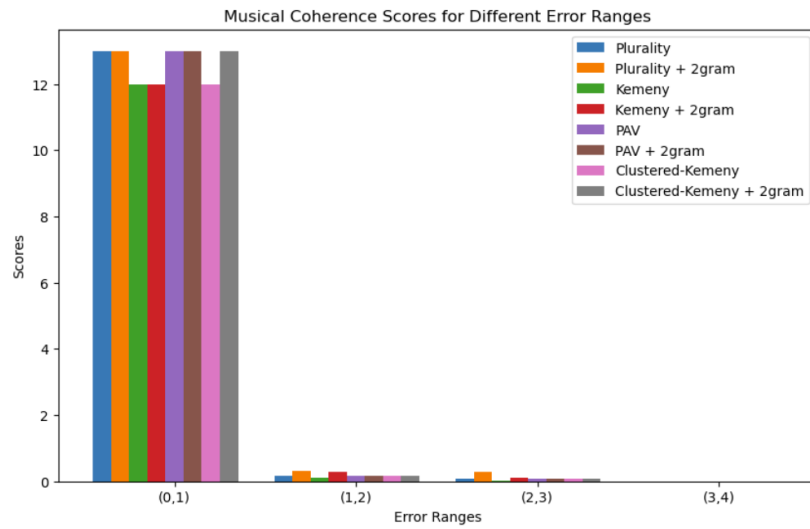


Fig. 5: Musical Coherence vs Error Ranges for 32 Agents.